PromiseTracker v3 product specification

Draft for team review, 10 November 2026.

This document explains how PromiseTracker v3 will work, in plain terms. The technical detail lives in the full system specification, linked at the end.

What PromiseTracker is

PromiseTracker records what politicians and public institutions promise, checks each promise against its source, and follows what happens to it over time. The public gets one place to see who promised what, whether it happened, and the evidence either way.

Why we are rebuilding it

The current version runs, but it has four problems we cannot patch.

It depends on two outside services. Source documents and politician lists live in Airtable, and published promises live in Meedan Check. If either service is unavailable, changes its pricing or changes its product, PromiseTracker stops working.

It does not track anything. A promise has a status, but no record of how that status changed. The timeline on a promise page is empty.

The public site will not cope with real traffic. Every page is built from scratch on every visit, the promise list loads everything at once, and the search box does nothing.

The country setup is confusing. Each country runs as a separate mini-site on its own web address, which makes it hard to manage and impossible to compare countries.

The product has not launched yet, so we can fix all four without moving any data.

How it works

The simplest way to understand v3 is to follow one promise through it.

1. Set up a country

An administrator adds a country by entering its name and code. The country gets its own section of the site straight away, at an address like `promisetracker.org/ke`. Nobody needs to deploy

anything or call a developer.

All countries share one site. That is the biggest structural change from the current version, and it is what makes cross-country comparison possible later.

2. Add the people and institutions

Editors add the actors whose promises we track. There are four kinds:

- **People**, such as a president, governor or minister
- **Parties**
- **Institutions**, such as a ministry or a county government
- **Places**, such as a county or a constituency

Any of the four can make promises. A manifesto belongs to a party, a budget commitment belongs to a ministry, and a campaign pledge belongs to a person.

A person's record keeps their full history of offices, so when a governor becomes a cabinet secretary we keep both. The current version can hold only one position and loses the rest.

Each actor gets a clear address by kind, such as `/ke/politicians/william-ruto` or `/ke/places/nairobi`.

3. Bring in the source material

Promises come from documents: manifestos, speeches, budgets, policy papers. Editors can bring them in three ways.

- Upload a file. PDF, Word, Excel, PowerPoint and most other formats work.
- Paste a link, and the system fetches the document.
- Import a spreadsheet. If a partner has already compiled a list of promises, each row becomes a promise directly. The editor matches the columns, previews the result, and confirms before anything is saved.

Every document shows where it is in the process, whether waiting, being read, ready for review or failed, along with the reason if something went wrong. Today that feedback only exists as notes written into Airtable.

4. Find the promises

Reading a 200-page manifesto for promises is slow. AI does the first pass. It reads the document and proposes candidate promises, each with the exact quote it found.

Two rules keep this honest. The system discards any candidate whose quote does not appear word for word in the document, so the AI cannot invent a promise. And a person reviews every

candidate before it goes further. The reviewer sees the quote highlighted in the original text and accepts, edits or rejects it.

AI is optional. A team that prefers to work without it can enter promises by hand, and nothing else changes.

5. Verify

Every accepted promise goes to a fact-checker, who confirms it is accurately quoted and attributed. This can happen in one of two places:

- In Superdesk, for newsrooms that already work there. The promise appears in the fact-checking desk, and the verdict flows back to PromiseTracker when it is published.
- Inside PromiseTracker itself, using a built-in review queue. This needs no other system.

A promise that fails verification is never published. We keep it on record, because a pattern of failures tells us something about the source or about the AI.

6. Publish

Verification and publishing are separate steps. A promise can be verified but still waiting on a written summary, and it should not go live half-finished.

Each record has one setting an editor controls: ready, on hold, or archived. A promise appears on the public site only when it is verified, marked ready, and everything above it is ready too. That last part matters. Putting a country, a politician or a source document on hold hides every promise underneath it within seconds. Releasing it brings them back, except for any that an editor had held individually.

If a manifesto turns out to be fake, one action removes every promise that came from it.

The system deletes nothing on its own. Archived records stay available for audit.

7. Track what happens

This is the feature the product is named for, and the current version does not have it.

When something happens to a promise, an editor adds an update with four parts: the date it happened, the status as of that date, a short account of what happened, and the evidence. A status might be "in progress", "kept" or "broken".

Nobody types in a promise's current status. The system takes it from the most recent published update. The status, its date and its evidence therefore always agree, and the public page shows the full history as a timeline.

8. Hear from citizens

Each promise page has a form where anyone can send an update, a correction or a promise we missed. Submissions go into a moderation queue and are never shown publicly as they arrive. When a moderator accepts one, it becomes a proper update on the timeline, credited to the contributor if they agreed to that.

What the public sees

A country page with a map, headline numbers and the politicians and institutions being tracked.

A profile page for each politician, party, institution or place. It opens with a scorecard showing how many promises are kept, in progress, broken or not started, followed by the full list. For a person it shows their offices and where they are from.

A promise page with the exact quote, a link to the source, the current status, the full timeline with evidence, who verified it and when, and a map where the promise applies to a specific place.

Lists that stay usable. Promise lists are paginated and can be filtered by status, topic, person and date. A filtered view has its own address, so it can be shared or bookmarked.

Search that works, across promises, people, parties, institutions and places.

English and French, with a visible language switch.

The site is built to stay fast under election-day traffic and to be found by search engines. Verified promises carry the standard fact-check markup that Google and others read.

Sharing the work

Three things let the content travel beyond our own site.

- **Embeds.** A partner newsroom can place a promise card or a politician's scorecard inside its own article.
- **A public data feed** for journalists and researchers, covering published content only.
- **Spreadsheet export** of promises and their updates.

Who does what

Role	What they do
Contributor	Adds documents, actors and draft promises. Cannot publish

Role	What they do
Reviewer	Reviews AI candidates and verifies promises
Editor	Decides what goes public and owns the promise timeline
Moderator	Handles citizen submissions
Administrator	Manages users, settings and connections to other systems

One person can hold several roles.

It runs on its own

The core of v3 is a complete product with no outside service attached. A team can upload documents, enter and verify promises, publish and track them using PromiseTracker alone.

Everything else is an optional add-on that can be switched on, replaced or removed without touching the core:

- Superdesk, for verification in the newsroom
- An AI provider, for finding candidate promises. We can change provider without rebuilding anything
- A document reader for PDF, Word and other complex formats
- Airtable, for any team that still wants to feed documents in from it

Meedan Check is no longer required. The design leaves room to connect it again if we ever need to.

What v3 leaves out

We chose not to build these now. The design keeps the door open for each.

- Separate permissions per country. Any editor can edit any country at launch. This must change before a partner newsroom gets an account, and it is first on the list for the second phase.
- A separate web address or visual theme per country
- Accounts for members of the public, such as following a promise or getting alerts
- Automatic detection of promises from news coverage
- Verification by the crowd. Citizens send evidence, and a fact-checker decides.

How we will roll it out

Phase 1, the core. Countries, actors, promises, updates and the timeline. Document upload, spreadsheet import, built-in verification, citizen submissions, and the public site with search. At the end of this phase PromiseTracker is usable with no outside service.

Phase 2, scale and assist. Reading PDF and Word documents, AI candidate finding, maps, embeds, the public data feed, export, and per-country permissions.

Phase 3, the newsroom. Verification in Superdesk. This comes last because it depends on setting up the newsroom's Superdesk, and nothing before it should wait on that.

What we need before launch

- An agreed list of promise statuses, each placed in one of five groups: not started, in progress, kept, broken, inconclusive. Every scorecard depends on this.
- An agreed list of topics, around 8 to 15.
- The first country, and the politicians we will track in it.
- For phase 3, someone with administrator access to our Superdesk.

The full specification

The full system specification covers the data model, the Superdesk connection, security, testing and operations. It is in the project repository on the `ft/v3-specification` branch:

https://github.com/CodeForAfrica/PromiseTracker/blob/ft/v3-specification/docs/spec/promisetracker-v3-system-specification.md